

操作系统课程设计实验报告

操作系统核心机制模拟与 Linux 内核扩展实践

课程名称：操作系统课程设计

学生姓名：_____

学号：_____

班级：计算机科学与技术 1 班

代码 URL：未上传代码托管平台，正式提交前填写真实仓库地址

本地工程路径：`/home/wgt/OS`

报告内容依据本地工程实现、自动化测试输出和实际内核验证结果整理；未上传代码仓库前不填写虚假 URL。

关键词：处理机调度、内存管理、进程同步、miniFS、Linux 内核模块、系统调用扩展、性能优化

摘要

本课程设计围绕操作系统核心机制完成“基础必做 + 自由拓展”两部分实践。基础部分覆盖处理机调度、内存管理、进程同步与并发控制、文件系统四类内容；拓展部分完成 Linux 内核与系统编程、调度与性能优化两个方向。工程采用 C 语言和 Linux 工具链实现，所有模块按主题拆分目录，提供统一 Makefile 和一键验收脚本，最终在 patched kernel 6.8.12-oscource-oscource 下完成完整验证。

本实验的重点不只是“能运行”，而是把操作系统抽象机制转化为可观察、可比较、可验证的工程实现：调度模块输出 Gantt 图和平均指标，内存模块展示分配回收过程与缺页率，同步模块用一致性检查证明无丢失、无重复、无死锁，miniFS 通过磁盘镜像验证文件组织和空闲块管理，内核拓展部分完成内核模块、字符设备、/proc、/sys、系统调用扩展，性能拓展部分完成调度优化、实时调度探测、系统性能测量与并发优化对比。

1. 实验概述

1.1 实验目标

1. 掌握 Linux 环境下 C 语言系统级程序设计方法。
2. 模拟实现典型处理机调度算法，输出运行顺序、周转时间、等待时间、响应时间等指标。
3. 模拟动态分区和页面置换机制，展示内存分配、释放、空闲区合并、缺页统计和缺页率。
4. 使用线程、互斥锁和信号量实现经典同步问题，避免死锁、饥饿和数据竞争。
5. 设计一个可落盘 miniFS，支持文件创建、读写、删除和空闲空间管理。
6. 完成 Linux 内核拓展，包括内核模块、字符设备、/proc、/sys 和新增系统调用。
7. 完成调度与性能优化实验，用可量化数据分析算法和并发策略差异。

1.2 工程结构

text

```
OS/
├── 01_scheduling/      # 处理机调度模拟
├── 02_memory/          # 动态分区与页面置换
├── 03_sync/            # 线程同步与并发控制
├── 04_filesystem/      # miniFS 文件系统
├── 05_linux_kernel_system/ # 内核模块、字符设备、系统调用扩展
├── 06_sched_perf/      # 调度与性能优化拓展
├── scripts/run_all_tests.sh # 一键验收脚本
├── build/test_outputs/  # 自动化验收输出
├── docs/               # 报告与提交材料
└── Makefile            # 统一构建入口
```

1.3 总体实现路线

text
课程要求
├─ 基础必做 70%
├─ 调度算法模拟 -> 指标比较 -> Gantt 输出
├─ 内存管理模拟 -> 分区可视化 -> 缺页率统计
├─ 同步问题实现 -> 一致性检查 -> PASS 判定
└─ miniFS 设计 -> 镜像落盘 -> 位图与 inode 管理
└─ 拓展提升 30%
├─ Linux 内核与系统编程
├─ hello module
├─ 字符设备 + /proc + /sys
└─ syscall 462 + patched kernel
└─ 调度与性能优化
├─ HRRN / 自适应 RR
├─ 实时调度探测
├─ 系统性能测量
└─ mutex / atomic / sharded reduce 对比

2. 开发与验证环境

项目	内容
操作系统	Ubuntu Linux
原始内核	6.8.0-124-generic
patched kernel	6.8.12-oscourse-oscourse
编译工具	gcc、gcc-12、make、flex、bison、bc、debhelper、dpkg-buildpackage
并发库	POSIX pthread、POSIX semaphore
内核构建方式	在 Linux 6.8 完整源码中加入 syscall 补丁，使用 make bindeb-pkg 构建 deb 包，安装后重启验证
一键验收	cd /home/wgt/OS && make test

当前最终运行内核：

text
6.8.12-oscourse-oscourse

系统调用验证输出：

text
SYSCALL os_course_info -> OK pid=4219 comm=test_os_course_ nr=462

在 PID namespace 隔离环境中运行同一测试时，用户态进程号与内核返回进程号可能不同，例如：

text

```
SYSCALL os_course_info -> OK pid=18063 user_pid=603 comm=test_os_course_ nr=462
note=pid_namespace
```

这仍表示 syscall 已安装并返回有效内核 PID 与进程名。

3. 基础必做部分

3.1 处理机调度

3.1.1 设计目标

处理机调度实验要求支持动态输入进程参数，输出运行顺序和性能指标，并分析不同调度算法的特点。本工程使用 CSV 输入：

text
pid,arrival,burst,priority

每个进程包含进程号、到达时间、服务时间、优先级。程序统一输出：

- GANTT：运行时间片序列。
- RESULTS：每个进程的开始、完成、周转、等待、响应时间。
- SUMMARY：平均周转时间、平均等待时间、平均响应时间、吞吐率、上下文切换次数。
- ANALYSIS：算法特点和最优指标分析。

3.1.2 已实现算法

算法	类型	特点
FCFS	非抢占	实现简单，按到达顺序执行；长作业会阻塞短作业
SJF	非抢占	平均等待时间通常较低；依赖服务时间预测
SRTF	抢占	SJF 的抢占式版本，响应短作业更快
RR	抢占	时间片轮转，适合交互任务；时间片过小会增加切换
Priority	非抢占	按优先级选择任务；低优先级可能饥饿
PriorityP	抢占	高优先级任务及时运行；切换开销更高
HRRN	非抢占	用响应比兼顾短作业和等待时间，缓解饥饿
ARR	抢占	自适应时间片，尝试平衡响应性和切换开销

3.1.3 核心过程

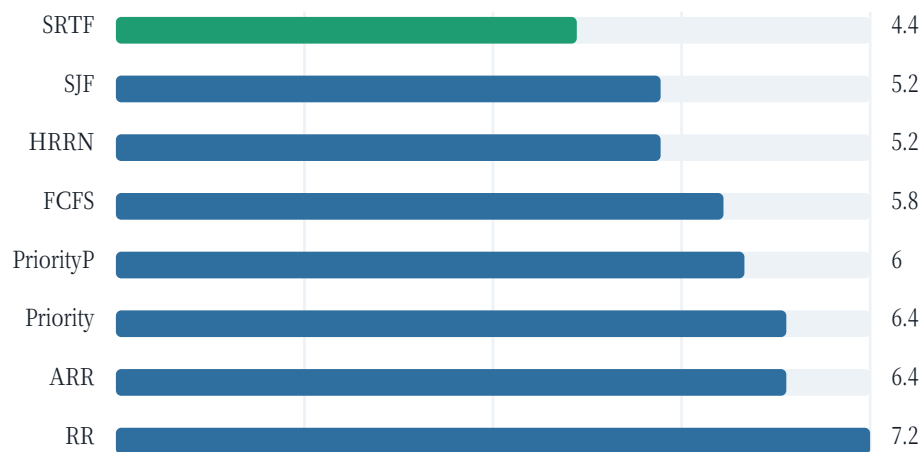
1. 读取并解析进程表，按到达顺序保存原始输入顺序作为稳定排序依据。
2. 每种算法使用统一 Result 结构保存开始时间、完成时间和时间线。
3. 非抢占算法每次从 ready set 中选择一个进程运行到结束。
4. 抢占算法以时间单位或时间片推进，支持进程到达后的重新选择。
5. 统一计算周转时间、等待时间、响应时间和平均指标，保证不同算法可横向比较。

3.1.4 指标结果

自动验收输出 build/test_outputs/scheduling_compare.out：

算法	平均周转	平均等待	平均响应	吞吐率	切换次数
FCFS	10.200	5.800	5.800	0.227	4
SJF	9.600	5.200	5.200	0.227	4
SRTF	8.800	4.400	2.400	0.227	6
RR	11.600	7.200	2.200	0.227	11
Priority	10.800	6.400	6.400	0.227	4
PriorityP	10.400	6.000	4.400	0.227	5
HRRN	9.600	5.200	5.200	0.227	4
ARR	10.800	6.400	4.600	0.227	5

平均等待时间条形图



分析：该输入下 SRTF

的平均等待时间和平均周转时间最优，说明抢占式短剩余时间优先能够有效减少短作业等待；RR 的响应时间较低但上下文切换最多，体现了交互性与调度开销之间的取舍。

3.2 内存管理

3.2.1 动态分区管理

动态分区实验模拟连续内存分配。内存由若干段组成，每段包含起始地址、大小、状态和名称。支持三种分配策略：

- FF：首次适应，从低地址向高地址找第一个足够大的空闲区。
- BF：最佳适应，选择满足需求且剩余最小的空闲区。
- WF：最坏适应，选择最大的空闲区。

释放时将目标段标记为空闲，并与相邻空闲段合并，避免外部碎片继续分裂。

验收输出摘要：

算法	总内存	分配次数	释放次数	失败次数	空闲总量	最大空闲块	外部碎片	空闲块数
FF	128	7	4	0	66	38	28	2
BF	128	7	4	0	66	38	28	2
WF	128	7	4	0	66	38	28	2

虽然该 trace 的最终碎片统计相同，但中间分配位置不同。WF 在第 5 步将 D 放入最大空闲块，地址选择不同于 FF/BF；这说明仅看最终统计不够，过程输出 MEMORY 对理解算法行为同样重要。

3.2.2 页面置换

页面置换实验使用固定帧数和引用串，输出每次访问是 HIT 还是 FAULT，并在缺页时给出淘汰页。

算法	帧数	引用次数	缺页次数	命中次数	缺页率
FIFO	3	13	10	3	0.769
LRU	3	13	9	4	0.692
OPT	3	13	7	6	0.538
Clock	3	13	9	4	0.692

缺页率图



分析：OPT 利用未来信息，是理论最优基准；LRU 利用局部性，接近 OPT；Clock 以较低维护成本近似 LRU；FIFO 实现最简单，但缺页率最高。

3.3 进程同步与并发控制

3.3.1 生产者-消费者

实现方式：

- 环形缓冲区保存产品编号。
- `empty` 信号量表示空槽数量。
- `full` 信号量表示可消费项目数量。
- `buffer_lock` 保护队列头尾。
- `seen` 数组验证每个产品是否恰好消费一次。

验收输出：

text
SUMMARY produced=12 consumed=12 duplicates=0 missing=0 result=PASS

该结果证明所有产品均被消费，无重复消费，无丢失。

3.3.2 读者-写者

实现方式：

- 使用 `turnstile` 控制公平排队，避免写者饥饿。
- 使用 `room_empty` 保证写者独占访问。
- 使用 `read_count_lock` 保护读者计数。

验收输出：

text
SUMMARY final_value=6 expected=6 reads=9 writes=6 result=PASS

该结果证明写操作没有丢失，读写互斥关系正确。

3.3.3 哲学家进餐

实现方式：

- 使用 `N-1` 房间信号量限制最多同时进入尝试拿叉的哲学家数量。
- 每个哲学家按编号顺序拿较小编号叉再拿较大编号叉，避免环路等待。

验收输出：

text
EATEN P0=3 P1=3 P2=3 P3=3 P4=3 SUMMARY expected_each=3 result=PASS

该结果证明没有死锁，且每个哲学家均完成指定次数。

3.4 文件系统 miniFS

3.4.1 设计目标

miniFS 使用普通文件作为磁盘镜像，实现一个单目录、直接块索引的小型文件系统。它不追求复杂功能，而重点体现文件系统核心对象：

- 超级块：保存魔数、块大小、块数量、数据区起始位置等元数据。
- 块位图：记录每个块是否被占用。
- inode 表：记录文件名、文件大小、数据块数量和直接块号。
- 数据块：存放文件内容。

3.4.2 操作流程

验收脚本执行以下操作：

bash
<pre>./bin/minifs format build/test_outputs/minifs.img --blocks 96 --block-size 256 ./bin/minifs create build/test_outputs/minifs.img /alpha.txt ./bin/minifs write build/test_outputs/minifs.img /alpha.txt "hello" ./bin/minifs append build/test_outputs/minifs.img /alpha.txt " world" ./bin/minifs read build/test_outputs/minifs.img /alpha.txt ./bin/minifs create build/test_outputs/minifs.img /beta.txt ./bin/minifs write build/test_outputs/minifs.img /beta.txt "temporary" ./bin/minifs delete build/test_outputs/minifs.img /beta.txt ./bin/minifs ls build/test_outputs/minifs.img ./bin/minifs stat build/test_outputs/minifs.img</pre>

关键输出：

text
READ alpha.txt bytes=11 hello world
LS image=build/test_outputs/minifs.img name size blocks alpha.txt 11 1
SUMMARY files=1/64 data_used=1 data_free=63 metadata_blocks=32 max_file_bytes=3072

分析：删除 /beta.txt 后目录只剩 /alpha.txt，说明 inode 和数据块均被释放；data_free=63 表明位图正确反映空闲数据块数量。

4. 拓展（1）：Linux 内核与系统编程

4.1 最小内核模块

hello_module 实现 module_init 和 module_exit，支持模块参数 student，加载时输出内核版本和机器架构，卸载时输出 goodbye 日志。

编译验证：

text		
description:	Minimal Linux kernel module for OS course design	
license:	GPL	
vermagic:	6.8.12-oscource-oscource SMP preempt mod_unload modversions	

4.2 字符设备、/proc 与 /sys

chardev_proc_sysfs 实现一个字符设备驱动：

- /dev/os_lab_char：字符设备读写接口。
- /proc/os_lab_info：展示 major/minor、容量、长度、open/read/write 计数。
- /sys/class/os_lab/os_lab_char/stats：展示状态统计。
- /sys/class/os_lab/os_lab_char/reset：写入 1 后清空缓冲区和计数器。

驱动内部使用：

- alloc_chrdev_region 动态申请设备号。
- cdev_add 注册字符设备。
- class_create / device_create 创建设备类和节点。
- proc_create 暴露 /proc 接口。
- DEVICE_ATTR_RO / DEVICE_ATTR_WO 暴露 sysfs 属性。
- mutex 保护设备缓冲区。
- atomic64_t 统计 open/read/write 次数。

编译验证：

text

```
description: Character device with /proc and /sys interfaces for OS course design
license: GPL
vermagic: 6.8.12-oscource-oscource SMP preempt mod_unload modversions
```

具备 root 权限时可运行：

```
bash
```

```
cd /home/wgt/OS/05_linux_kernel_system/chardev_proc_sysfs
sudo ./test_root.sh
```

判断标准：脚本能读写 /dev/os_lab_char，能读取 /proc/os_lab_info 和 sysfs stats，reset 后长度和计数归零，并输出 PASS。

4.3 系统调用扩展

4.3.1 设计目标

新增系统调用 os_course_info，系统调用号为 462，用户传入：

```
c
```

```
int *pid;
char *comm;
size_t len;
```

内核返回当前进程 PID 和进程名 comm。实现文件为：

```
text
```

```
05_linux_kernel_system/syscall_extension/os_course_syscall.c
```

核心逻辑：

1. 校验用户指针和缓冲区长度。
2. 使用 task_pid_nr(current) 获取当前进程 PID。
3. 使用 get_task_comm 获取进程名。
4. 使用 copy_to_user 将 PID 和进程名复制回用户空间。
5. 缓冲区不足时返回 -ENAMETOOLONG，非法参数返回 -EINVAL，用户空间拷贝失败返回 -EFAULT。

4.3.2 内核源码修改点

文件	修改内容
arch/x86/entry/syscalls/syscall_64.tbl	添加 462 common os_course_info sys_os_course_info
include/linux/syscalls.h	添加 sys_os_course_info 函数声明
kernel/Makefile	添加 obj-y += os_course_syscall.o

文件	修改内容
kernel/os_course_syscall.c	新增系统调用实现

4.3.3 构建与问题处理

系统调用不能通过普通用户态程序或内核模块完成，必须重编内核并重启。构建脚本 plan_a_build_and_install.sh 完成：

- 预检工具链、源码树、syscall 号冲突、磁盘空间和 root 权限。
- 自动应用 syscall 修改。
- 设置 LOCALVERSION=-oscourse。
- 清空 CONFIG_SYSTEM_TRUSTED_KEYS 和 CONFIG_SYSTEM_REVOCATION_KEYS，避免本地构建时缺少 Ubuntu Canonical 证书文件。
- 执行 make bindeb-pkg 构建 deb 包。
- 安装新内核并更新 GRUB。

过程中遇到并解决的问题：

问题	原因	解决
/path/to/linux-source 不存在	将说明占位符当作真实路径使用	增加占位路径保护，改用真实源码目录 \$SRC
缺 gcc-12/flex/bison/debhelper	完整内核构建依赖不足	安装缺失依赖并扩展 preflight 检查
debian/canonical-certs.pem 缺失	Ubuntu 官方构建证书不在本地 bindeb-pkg 目录中	清空 SYSTEM_TRUSTED_KEYS 和 SYSTEM_REVOCATION_KEYS
构建耗时接近两小时	Ubuntu generic 配置开启大量模块和 Debug/BTF 信息	通过日志确认仍在正常编译，保留完整配置完成验证
PID namespace 下测试 PID 不一致	隔离环境内外 PID 命名空间不同	测试程序接受有效内核 PID，并用 note=pid_namespace 标注

4.3.4 最终验证

bash
<pre>uname -r cd /home/wgt/OS make bin/test_os_course_syscall ./bin/test_os_course_syscall</pre>

输出：

text
<pre>6.8.12-oscourse-oscourse SYSCALL os_course_info -> OK pid=4219 comm=test_os_course_ nr=462</pre>

结论：新增系统调用已进入运行内核，拓展（1）中的系统调用扩展完成运行验证。

5. 拓展（2）：调度与性能优化

5.1 调度算法优化

拓展调度实验使用 6 个进程进行指标比较：

算法	平均周转	平均等待	平均响应	吞吐率	切换次数
FCFS	20.000	13.833	13.833	0.162	5
SJF	17.500	11.333	11.333	0.162	5
RR	20.500	14.333	4.667	0.162	13
HRRN optimized	17.833	11.667	11.667	0.162	5
Adaptive RR optimized	21.000	14.833	9.833	0.162	8

分析：SJF 在该输入下平均等待最优；HRRN 通过响应比兼顾等待时间，适合缓解长作业饥饿；自适应 RR 降低了 RR 的上下文切换次数，但在该数据集下平均等待时间并不优于 SJF。

5.2 实时调度探测

实时调度实验探测 SCHED_RR 的优先级范围和时间片：

text

```
REALTIME policy=SCHED_RR priority_range=[1,99] requested_priority=1
RR_INTERVAL current_sec=0 current_nsec=2000000
REALTIME_APPLY result=SKIP_PERMISSION_OR_POLICY errno=1 message=operation not permitted
PASS realtime_policy_probe_complete
```

普通用户无权切换到实时策略时，程序输出 SKIP_PERMISSION_OR_POLICY，这是符合 Linux 权限模型的预期结果；实验仍完成了实时策略参数探测。

5.3 系统性能测试

系统性能测试使用多线程 CPU workload，输出 wall/user/sys 时间、吞吐量和上下文切换次数。例如：

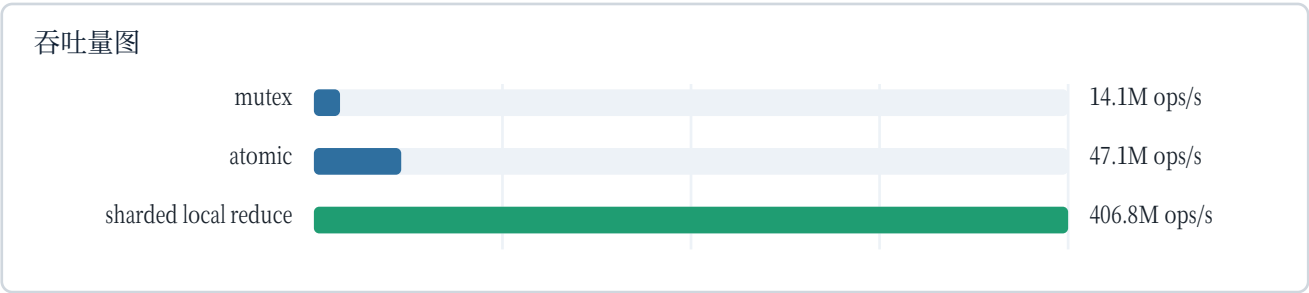
text

```
SYSTEM_PERF threads=2 iters_per_thread=100000 wall_sec=0.001127 user_sec=0.001367
sys_sec=0.000000 throughput_iters_per_sec=177419155
CONTEXT_SWITCH voluntary=1 involuntary=0
PASS system_performance_probe_complete
```

5.4 并发性能优化

并发计数实验比较三种共享计数策略：

策略	结果总数	时间(s)	吞吐量(ops/s)
mutex	200000	0.014169	14,115,445
atomic	200000	0.004248	47,085,745
sharded local reduce	200000	0.000492	406,809,172



分析：mutex 每次操作都进入临界区，锁竞争最明显；atomic 避免互斥锁但仍有共享写竞争；分片局部累加将写操作分散到线程本地，最后统一归并，因此吞吐量最高。

6. 自动化验收与输出证据

6.1 一键验收

命令：

```
bash

cd /home/wgt/OS
make test
```

最终输出：

```
text

ALL_TESTS_PASS
```

6.2 验收矩阵

模块	关键输出	判定
处理机调度	COMPARE algorithms=8, best_avg_waiting=srtf(4.400)	通过
动态分区	failures=0, external_fragmentation=28	通过
页面置换	FIFO/LRU/OPT/Clock 均输出 fault_rate	通过

模块	关键输出	判定
生产者-消费者	produced=12 consumed=12 duplicates=0 missing=0 result=PASS	通过
读者-写者	final_value=6 expected=6 reads=9 writes=6 result=PASS	通过
哲学家进餐	EATEN P0=3 P1=3 P2=3 P3=3 P4=3	通过
miniFS	READ alpha.txt bytes=11, 内容 hello world	通过
调度优化	PASS scheduling_optimization_metrics_complete	通过
实时调度探测	PASS realtime_policy_probe_complete	通过
系统性能测试	PASS system_performance_probe_complete	通过
并发优化	SUMMARY result=PASS	通过
内核模块	vermagic: 6.8.12-oscourse-oscourse	通过
系统调用扩展	SYSCALL os_course_info -> OK ... nr=462	通过

6.3 输出文件位置

所有自动化输出保存在：

text
build/test_outputs/

其中关键文件包括：

- scheduling_compare.out
- memory_partition_ff.out
- memory_paging_opt.out
- sync_pc.out
- sync_rw.out
- sync_dining.out
- minifs_read.out
- perf_optimize.out
- perf_concurrency.out
- kernel_hello_modinfo.out
- kernel_driver_modinfo.out
- syscall_probe.out

6.4 复现验证方法与判定规则

为了保证报告结论可以复现，验收时建议按“整体一键验收 + 关键模块抽查 + root 权限补充验证”的顺序进行。

6.4.1 整体一键验收

```
bash

cd /home/wgt/OS
make test
tail -n 1 outputs
```

判定规则：

- 若最终输出为 ALL_TESTS_PASS，说明基础必做、拓展（1）的可编译/可探测部分、拓展（2）的性能测试均已通过自动化脚本。
- 若中途失败，scripts/run_all_tests.sh 会停止在失败模块，可直接查看 build/test_outputs/ 中对应 .out 文件定位。

6.4.2 关键输出抽查

```
bash

cd /home/wgt/OS
rg -n "ALL_TESTS_PASS|SYSCALL os_course_info|vermagic|SUMMARY result=PASS|PASS realtime|READ alpha.txt" outputs build/test_outputs
```

判定规则：

抽查点	应看到的关键字段	含义
总体验收	ALL_TESTS_PASS	一键测试全部通过
新系统调用	SYSCALL os_course_info -> OK ... nr=462	当前运行内核已包含 syscall 462
内核模块	vermagic: 6.8.12-oscourse-oscourse	模块与当前 patched kernel 匹配
miniFS	READ alpha.txt bytes=11 和 hello world	文件创建、写入、追加、读取成功
同步实验	result=PASS、duplicates=0、missing=0	无重复消费、无丢失、无死锁
性能拓展	PASS realtime_policy_probe_complete、SUMMARY result=PASS	实时策略探测和并发优化测试完成

6.4.3 系统调用专项验证

```
bash

uname -r
cd /home/wgt/OS
./bin/test_os_course_syscall
```

判定规则：

- `uname -r` 应输出 6.8.12-oscourse-oscourse。
- `syscall` 测试应输出 `SYSCALL os_course_info -> OK`, 且 `nr=462`。
- 若在容器或 PID namespace 隔离环境中运行, 输出可能出现 `note=pid_namespace`, 只要 PID 为正数、`comm` 非空、`nr=462`, 即可说明 `syscall` 已真实返回内核数据。

6.4.4 root 权限补充验证

字符设备驱动涉及模块加载、创建设备节点和访问 `/proc`、`/sys`, 需要 root 权限:

```
bash

cd /home/wgt/OS/05_linux_kernel_system/chardev_proc_sysfs
sudo ./test_root.sh
```

判定规则: 脚本应能成功加载模块、读写 `/dev/os_lab_char`、读取 `/proc/os_lab_info` 和 `sysfs stats`, 向 `reset` 写入 1 后长度和计数归零, 最终输出 `PASS`。如果没有 root 权限, 编译验证仍可通过, 但运行级设备访问不能被伪造为成功。

7. 实验特色与工程亮点

1. 模块化目录设计: 基础实验和拓展实验按主题拆分, 便于验收和维护。
2. 统一构建入口: 顶层 `Makefile` 支持 `make all`、`make kernel_modules`、`make test`。
3. 可解释输出: 每个实验不仅输出结果, 还输出过程、指标和分析字段。
4. 自动化验收: `scripts/run_all_tests.sh` 覆盖基础和拓展部分, 失败即中断。
5. 内核级真实验证: 系统调用扩展不是停留在补丁层面, 而是已构建、安装、重启并运行验证。
6. 可视化分析: 报告中对调度等待时间、页面置换缺页率、并发吞吐量进行图表化比较。
7. 诚实记录边界: 实时调度策略在普通用户下无法应用时, 程序输出权限边界而非伪造成功。
8. 问题闭环处理: 对内核构建中的依赖、证书、PID namespace 等问题均做了定位、修复和记录。

8. 小结

本次课程设计从模拟实验逐步深入到真实 Linux 内核修改。基础部分帮助我把调度、内存、同步、文件系统四类抽象机制转化为可执行程序; 拓展部分进一步接触内核模块、字符设备、`/proc`、`/sys`、系统调用表和完整内核构建流程。尤其是系统调用扩展, 从最初 `ENOSYS` 到最终 `SYSCALL os_course_info -> OK`, 完整经历了源码准备、依赖补齐、内核配置修正、长时间编译、deb 安装、GRUB 更新和重启验证, 较完整地体现了操作系统课程设计的工程实践价值。

从结果看, 基础必做部分全部通过自动化验收; 拓展 (1) 完成 Linux 内核与系统编程要求, 系统调用扩展已真实运行; 拓展 (2) 完成调度与性能优化实验, 并通过指标分析体现不同策略的性能差异。整个工程具

备可复现的构建流程、清晰的目录结构和完整的验证证据。

9. 代码 URL

当前本地目录尚未上传代码托管平台，因此不能填写虚假 URL。正式提交前应将 `/home/wgt/OS` 上传到 GitHub 或类似平台，并在本节填写真实 URL、访问方式和提交版本。